

Préparation au Concours National Commun 2026 — Informatique MP

Épreuve d'Informatique — Énoncé

Filière MP — Durée : 4 heures — Quatre problèmes indépendants

*Autour de la blockchain Ethereum : algorithmique, arbres de Merkle, partition (programmation dynamique) et finance décentralisée*Auteur : **Pr Agrégé El Hadiq Zouhair**cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit. **Consignes.** L'épreuve comporte **quatre problèmes indépendants**. Les questions sont numérotées **en continu** sur l'ensemble du sujet ; le corrigé reprend la même numérotation. On pourra traiter les problèmes dans l'ordre de son choix.

Problème I — Algorithmique de la blockchain Ethereum

Présentation. Ethereum est un ordinateur mondial décentralisé. Ce problème étudie, de manière *largement indépendante* partie par partie, quelques structures et algorithmes qui le sous-tendent : la mesure des ressources (le *gaz*), l'encodage hexadécimal des adresses, les *arbres de Merkle*, la sélection aléatoire d'un validateur, et enfin un problème d'optimisation résolu par *programmation dynamique*.

Conventions.

- Le langage est Python 3. Les entiers Python sont de taille arbitraire.
- Pour une liste L , `len(L)` est sa longueur et $L[i]$ son terme d'indice i (à partir de 0).
- $\lfloor x \rfloor$ désigne la partie entière de x . $\log_b$ est le logarithme en base b .
- On rappelle que $n // b$ et $n \% b$ sont le quotient et le reste de la division euclidienne.

Les questions sont numérotées **en continu** sur tout le sujet.

Partie I — Unités et compteur de gaz

L'unité de compte d'Ethereum est l'*ether* ; sa plus petite subdivision est le *wei*, avec $1 \text{ ether} = 10^{18} \text{ wei}$. Chaque instruction exécutée consomme une quantité fixe de *gaz* ; une transaction fixe un plafond de gaz au-delà duquel l'exécution est interrompue.

Question 1. Écrire une fonction `ether_vers_wei(e)` qui, pour un entier $e \geq 0$, renvoie le nombre de wei correspondant à e ethers.

Question 2. Écrire `gaz_restant(plafond, couts)` où `couts` est la liste des coûts (entiers positifs) des instructions exécutées dans l'ordre. La fonction retranche les coûts un à un ; si, avant une instruction, le gaz disponible est *strictement* inférieur à son coût, l'exécution s'arrête et la fonction renvoie la chaîne "*panne de gaz*". Sinon elle renvoie le gaz restant.

Question 3. Un contrat exécute une boucle dont l'itération numéro i (pour $i \geq 0$) coûte $g/2^i$ unités de gaz, où g est un réel strictement positif. Montrer que le coût total $C = \sum_{i=0}^{+\infty} \frac{g}{2^i}$ est fini et vaut exactement $2g$. En déduire que, quel que soit le nombre d'itérations effectivement exécutées, le coût cumulé est majoré par $2g$.

Partie II — Encodage hexadécimal des adresses

Une adresse Ethereum est un entier affiché en base 16 (hexadécimal), avec l'alphabet "0123456789abcdef". On étudie l'encodage d'un entier en base 16.

Question 4. Écrire `entier_vers_hex(n)` qui, pour un entier $n \geq 0$, renvoie la chaîne de ses chiffres hexadécimaux (sans le préfixe 0x), avec la convention `entier_vers_hex(0) == "0"`.

Question 5. Écrire `hex_vers_entier(s)` qui, pour une chaîne `s` de chiffres hexadécimaux minuscules, renvoie l'entier qu'elle représente.

Question 6. Pour $n \geq 1$, montrer que le nombre de tours de la boucle de `entier_vers_hex(n)` (donc le nombre de chiffres produits) est exactement $\lfloor \log_{16} n \rfloor + 1$. En déduire la complexité temporelle de `entier_vers_hex` en fonction du nombre de chiffres.

Question 7. Justifier la **terminaison** de la boucle de `entier_vers_hex` à l'aide d'un *variant de boucle*, puis prouver la **correction**, c'est-à-dire que pour tout $n \geq 0$, `hex_vers_entier(entier_vers_hex(n))` est égal à n .

Partie III — Arbres de Merkle

Un *arbre de Merkle* résume un ensemble de données par un seul haché, la *racine*. On se donne une fonction de combinaison

$$\text{parent}(a, b) = (31a + b) \bmod 1000003,$$

et les feuilles sont des entiers. La racine se calcule par niveaux : à chaque niveau, on remplace la liste courante par la liste des `parent` de couples consécutifs ; si la liste a une longueur impaire, on **duplique son dernier élément** avant de l'appairer. On s'arrête lorsqu'il ne reste qu'un élément.

Question 8. Écrire `racine_merkle(feuilles)` qui prend une liste non vide de feuilles et renvoie la racine. Vérifier à la main que `racine_merkle([1, 2, 3, 4])` vaut 1120.

Question 9. Soit n le nombre de feuilles. Montrer que le nombre total d'appels à `parent` effectués par `racine_merkle` est un $O(n)$. *Indication* : majorer la somme des tailles des niveaux successifs.

Question 10. On veut prouver l'*inclusion* d'une feuille sans révéler tout l'arbre. Écrire `preuve_inclusion(feuilles, i)` qui renvoie la liste des couples (frère, côté) rencontrés en remontant de la feuille d'indice i jusqu'à la racine, où le côté vaut 'L' ou 'R' et indique si le frère est à gauche ou à droite. Écrire ensuite `verifier(feuille, preuve, racine)` qui recalcule la racine à partir de la feuille et de la preuve, et renvoie un booléen.

Question 11. Montrer par récurrence sur la hauteur de l'arbre que, si la fonction `parent` est *résistante aux collisions* (on ne sait pas produire $(a, b) \neq (a', b')$ avec `parent(a, b) == parent(a', b')`), alors une preuve d'inclusion acceptée par `verifier` garantit que la feuille figure bien parmi les feuilles ayant produit la racine. Quelle est la taille d'une preuve d'inclusion en fonction de n ?

Partie IV — Sélection aléatoire d'un validateur

À chaque créneau (*slot*), un validateur donné est choisi comme proposeur avec probabilité $p \in]0, 1[$, indépendamment des autres créneaux. On note N le numéro (à partir de 1) du premier créneau où ce validateur est choisi.

Question 12. Donner, pour $k \geq 1$, la probabilité $\mathbb{P}(N = k)$. Reconnaître la loi de N et en déduire l'espérance $\mathbb{E}[N]$.

Question 13. Montrer que $\mathbb{P}(N > k) = (1 - p)^k$, puis que $\mathbb{P}(N < +\infty) = 1$ (*terminaison presque sûre* : le validateur est choisi en temps fini avec probabilité 1).

Question 14. Application numérique : pour $p = 1/4$, donner $\mathbb{E}[N]$ et la probabilité $\mathbb{P}(N \leq 4)$ sous forme de fraction irréductible.

Partie V — Partition équilibrée par programmation dynamique

On dispose d'une liste S d'entiers strictement positifs. Le *problème de partition* demande s'il existe un sous-ensemble de S dont la somme vaut exactement la moitié de la somme totale (les deux parts ont alors la même somme). C'est un problème classique lié aux preuves à divulgation nulle.

Question 15. Montrer que si la somme totale $T = \sum S$ est impaire, la réponse est nécessairement *non*. On suppose désormais T paire et on pose $C = T/2$.

Question 16. Le *problème de décision* associé (« existe-t-il un sous-ensemble de somme C ? ») est **NP**. Justifier en décrivant un *certificat* et l'algorithme de vérification, ainsi que sa complexité.

Question 17. On définit, pour $0 \leq i \leq \text{len}(S)$ et $0 \leq c \leq C$, le booléen $A[i][c] =$ « il existe un sous-ensemble des i premiers éléments de S de somme c ». Donner la relation de récurrence vérifiée par $A[i][c]$ et son initialisation.

Question 18. En déduire une fonction `peut_partitionner(S)` utilisant un tableau à une dimension `dp` de taille $C + 1$. *Attention au sens de parcours* pour ne pas réutiliser deux fois le même élément. Donner sa complexité en temps et en espace, et expliquer le terme *pseudo-polynomial*.

Question 19. Prouver la correction de `peut_partitionner` en énonçant un *invariant* portant sur `dp` après le traitement des i premiers éléments, puis en le démontrant par récurrence sur i . Justifier enfin la terminaison.

Problème II — Arbres de Merkle et preuves d'inclusion

Présentation. Un *arbre de Merkle* permet de résumer une grande collection de données par un unique condensé, la *racine*, et de prouver qu'une donnée en fait partie à l'aide d'un *certificat* de taille logarithmique. C'est un outil central des blockchains (preuves d'inclusion, listes d'autorisation, stockage vérifiable). Dans tout le problème, les *feuilles* sont des entiers, et l'on combine deux nœuds par la fonction

$$\text{parent}(a, b) = (31a + b) \bmod p, \quad p = 1000003 \text{ (premier).}$$

Conventions. Le langage est Python 3. Pour une liste L , `len(L)` est sa longueur, $L[i]$ son terme d'indice i (≥ 0), $L[-1]$ son dernier terme. $\lceil x \rceil$ est la partie entière supérieure, $\log_2$ le logarithme binaire. Les questions sont numérotées **en continu**.

Partie I — La fonction de combinaison

Question 20. Écrire `parent(a, b)` conforme à la définition ci-dessus.

Question 21. Calculer à la main `parent(1, 2)`, `parent(3, 4)` et `parent(33, 97)`.

Question 22. On dit que `parent` est *déterministe*. Expliquer pourquoi ce caractère déterministe est indispensable pour que deux nœuds différents du réseau, partant des mêmes feuilles, calculent la même racine.

Question 23. On admet, pour la suite, que `parent` est *résistante aux collisions* : on ne sait pas exhiber deux couples distincts $(a, b) \neq (a', b')$ tels que `parent(a, b) = parent(a', b')`. Rappeler pourquoi une fonction $h : \mathbb{N}^2 \rightarrow \{0, \dots, p-1\}$ ne peut pas être *injective*, et donc pourquoi la résistance aux collisions est une hypothèse *calculatoire* (« difficile à violer ») et non une impossibilité mathématique.

Partie II — Construction de la racine

La racine se calcule par niveaux : tant qu'il reste plus d'un nœud, si le nombre de nœuds est impair on **duplique le dernier**, puis on remplace la liste par celle des `parent` de couples consécutifs.

Question 24. Écrire `racine_merkle(feUILles)` (liste non vide) renvoyant la racine.

Question 25. Dérouler le calcul de `racine_merkle([1, 2, 3, 4])` niveau par niveau et donner la valeur de la racine.

Question 26. Faire de même pour `racine_merkle([1, 2, 3])` en explicitant l'étape de duplication.

Question 27. Soit n le nombre de feuilles. En notant $t_0 = n$ et $t_{k+1} = \lceil t_k/2 \rceil$ la taille du niveau $k+1$, montrer que le nombre de niveaux est $O(\log n)$ et que le nombre total d'appels à `parent` est un $O(n)$.

Partie III — Preuves d'inclusion

Question 28. Écrire `preuve_inclusion(feUILles, i)` renvoyant la liste des couples (frère, côté), avec côté valant `'L'` ou `'R'`, rencontrés en remontant de la feuille d'indice i jusqu'à la racine.

Question 29. Écrire `verifier(feuille, preuve, racine)` qui recalcule la racine depuis la feuille et la preuve et renvoie un booléen. Vérifier sur l'exemple `[1, 2, 3, 4]`, $i = 0$.

Question 30. Montrer que la taille d'une preuve d'inclusion est le nombre de niveaux, soit $O(\log n)$, et commenter l'intérêt (comparer au coût de transmettre toutes les feuilles).

Question 31. Soundness. Démontrer par récurrence sur la hauteur h de l'arbre que, sous l'hypothèse de résistance aux collisions, si `verifier(feuille, preuve, racine)` renvoie `True` alors `feuille` figure effectivement parmi les feuilles ayant produit `racine`.

Partie IV — Hauteur et mise à jour

Question 32. Montrer que la hauteur de l'arbre (nombre de niveaux au-dessus des feuilles) vaut $H(n) = \lceil \log_2 n \rceil$ pour $n \geq 1$ (avec $H(1) = 0$). Donner $H(n)$ pour $n \in \{1, 2, 3, 4, 5, 8, 9\}$.

Question 33. Lorsqu'une seule feuille change de valeur, quels nœuds de l'arbre doivent être recalculés ? En déduire qu'une mise à jour de la racine coûte $O(\log n)$ appels à `parent` (et non $O(n)$). Écrire `maj_racine(feUILles, i, v)` qui renvoie la nouvelle racine lorsque la feuille d'indice i prend la valeur v , en **réutilisant** `racine_merkle`. *On ne demande pas ici l'implémentation optimale en $O(\log n)$, mais on justifiera la borne.*

Problème III — Le problème de partition et la programmation dynamique

Présentation. Le *problème de partition* consiste, étant donné une liste S d'entiers strictement positifs, à décider si l'on peut la séparer en deux sous-ensembles de même somme. Il apparaît dans les preuves à divulgation nulle de connaissance (chapitre « Zero-Knowledge Proofs » de *Mastering Ethereum*) comme exemple canonique de problème *facile à vérifier* mais *difficile à résoudre*. On note $T = \sum_{x \in S} x$ la somme totale.

Conventions. Langage Python 3 ; `sum(S)` est la somme des éléments, `len(S)` leur nombre. Le module `itertools` fournit `combinations(S, r)` qui énumère les sous-ensembles de S à r éléments. Questions numérotées **en continu**.

Partie I — Recherche exhaustive

Question 34. Écrire `demi_somme(S)` qui renvoie `None` si $T = \text{sum}(S)$ est impair, et $T/2$ (entier) sinon.

Question 35. Écrire `partition_exhaustive(S)` qui décide, en énumérant tous les sous-ensembles à l'aide de `itertools.combinations`, s'il existe un sous-ensemble de somme $T/2$. La fonction renvoie un booléen. On renverra `False` si T est impair.

Question 36. Combien de sous-ensembles possède un ensemble à n éléments ? En déduire que `partition_exhaustive` a une complexité temporelle en $\Theta(2^n)$ dans le pire des cas, et expliquer pourquoi cette approche est inutilisable pour n grand.

Partie II — Le problème de décision Subset-Sum

On suppose désormais T paire et on pose $C = T/2$. Le *problème de décision* Subset-Sum demande : « existe-t-il un sous-ensemble de S de somme exactement C ? »

Question 37. Décrire un *certificat* (une preuve courte) permettant de convaincre en temps polynomial qu'une instance est positive, et l'algorithme de *vérification* associé. Donner sa complexité.

Question 38. En déduire que Subset-Sum appartient à la classe **NP**. On rappelle qu'un *problème de décision* est dans **NP** s'il admet un *certificat vérifiable en temps polynomial*. On mentionnera, sans le démontrer, que Subset-Sum est en fait NP-complet.

Partie III — Récurrence et table de programmation dynamique

On définit, pour $0 \leq i \leq n$ (avec $n = \text{len}(S)$) et $0 \leq c \leq C$, le booléen

$$A[i][c] = \text{« il existe un sous-ensemble des } i \text{ premiers éléments de } S \text{ de somme } c \text{ »}.$$

Question 39. Justifier la relation de récurrence

$$A[i][c] = A[i-1][c] \vee (c \geq S[i-1] \wedge A[i-1][c - S[i-1]]) \quad (i \geq 1),$$

et préciser l'initialisation $A[0][\cdot]$.

Question 40. En déduire une fonction `subset_sum_2d(S, C)` qui construit la table A (de taille $(n+1) \times (C+1)$) et renvoie $A[n][C]$. Donner sa complexité en temps et en espace.

Partie IV — Version optimisée en espace

Question 41. Écrire `peut_partitionner(S)` qui, après avoir écarté le cas T impair, décide l'existence d'une partition équilibrée en n'utilisant qu'un tableau `dp` à **une dimension** de taille $C + 1$. *Indication* : parcourir c de C jusqu'à x en **décroissant** pour chaque élément x de S .

Question 42. Expliquer précisément pourquoi le parcours **décroissant** de c est nécessaire : que se passerait-il avec un parcours croissant ?

Question 43. Donner la complexité en temps et en espace de `peut_partitionner`, et expliquer le terme *pseudo-polynomial* (comparer C à la taille de codage de l'entrée).

Partie V — Reconstruction et correction

Question 44. À partir de la table A de la Partie III, écrire `sous_ensemble(S, C)` qui renvoie *un* sous-ensemble de somme C (sous forme de liste), ou `None` s'il n'en existe pas. *Indication* : remonter la table de $i = n$ à 1 ; l'élément $S[i-1]$ est pris ssi $A[i-1][C]$ est faux (le retirer impose de l'utiliser).

Question 45. Énoncer un *invariant* portant sur `dp` après le traitement des i premiers éléments dans `peut_partitionner`, puis le démontrer par récurrence sur i . Conclure sur la correction, et justifier la terminaison.

Problème IV — Finance décentralisée : teneurs de marché automatisés

Présentation. Un *teneur de marché automatisé* (*Automated Market Maker*, AMM) remplace le carnet d'ordres par une *réserve de liquidité* contenant deux jetons, en quantités x et y . Le prix est fixé par une formule et non par l'appariement d'ordres. Le modèle le plus répandu (Uniswap V2) impose que le *produit* xy reste constant : $xy = k$. Ce problème étudie ce mécanisme, ses bornes, puis un modèle de *prêt sur-collatéralisé*.

Conventions. Langage Python 3. Les réserves x, y sont des réels strictement positifs. On note $\Delta x > 0$ la quantité de jeton X apportée à la réserve et $\Delta y > 0$ la quantité de jeton Y reçue en échange (échange *sans frais*, sauf mention contraire). Questions numérotées **en continu**.

Partie I — Prix et produit constant

Question 46. Écrire `prix(x, y)` qui renvoie le prix d'une unité du jeton X exprimé en jeton Y , c'est-à-dire y/x . Vérifier que pour une réserve de 10 ETH et 100 USDC, le prix d'un ETH est 10 USDC.

Question 47. Un échange amène la réserve de (x, y) à $(x + \Delta x, y - \Delta y)$. En imposant la conservation du produit $(x + \Delta x)(y - \Delta y) = xy$, montrer que

$$\Delta y = \frac{y \Delta x}{x + \Delta x}.$$

Question 48. Écrire `sortie_amm(x, y, dx)` qui renvoie Δy . Vérifier que `sortie_amm(100, 100, 100)` vaut 50.

Partie II — Bornes et monotonie (anti-drainage)

Question 49. Montrer que pour tout $\Delta x > 0$ on a $\Delta y < y$: on ne peut jamais vider entièrement la réserve du jeton Y .

Question 50. Montrer que la fonction $\Delta x \mapsto \Delta y(\Delta x) = \frac{y \Delta x}{x + \Delta x}$ est **strictement croissante** sur $]0, +\infty[$ et que $\Delta y \rightarrow y$ lorsque $\Delta x \rightarrow +\infty$. Interpréter en termes de *glissement* (*slippage*) : plus l'échange est gros relativement à la réserve, plus le prix effectif se dégrade.

Question 51. Le *prix effectif* d'un échange est $\Delta x / \Delta y$. L'exprimer en fonction de $x, y, \Delta x$ et montrer qu'il est $> x/y$ (le prix « spot » inverse), avec égalité à la limite $\Delta x \rightarrow 0$. Conclure sur l'impossibilité de drainer la réserve à prix constant.

Partie III — Frais cumulés et série géométrique

On modélise une suite d'échanges dont les volumes décroissent géométriquement : le i -ième échange ($i \geq 0$) porte un volume $v_0 \rho^i$ avec $0 < \rho < 1$, et prélève des frais proportionnels de taux f .

Question 52. Montrer que le total des frais perçus $\Phi = \sum_{i=0}^{+\infty} f v_0 \rho^i$ est fini et vaut $\frac{f v_0}{1 - \rho}$.

Question 53. Application : pour $f = 0,003$, $v_0 = 1000$ et $\rho = 0,9$, calculer Φ .

Question 54. Écrire `frais_cumules(f, v0, rho, n)` qui calcule la somme *partielle* $\sum_{i=0}^{n-1} f v_0 \rho^i$, et donner une majoration de l'erreur $\Phi - \text{frais_cumules}(f, v_0, \rho, n)$ par une expression géométrique en ρ^n .

Partie IV — Prêt sur-collatéralisé et intérêts composés

Un emprunteur dépose un *collatéral* de valeur G et emprunte un principal P . Le taux de collatéralisation exigé est $\tau\%$: l'emprunt n'est autorisé que si $G \geq P\tau/100$. La dette croît ensuite par intérêts composés : après n périodes au taux r par période, la dette vaut $P(1+r)^n$. La position est *liquidée* dès que la dette dépasse le collatéral G .

Question 55. Écrire `collateral_requis(P, tau)` qui renvoie le collatéral minimal $\lfloor P\tau/100 \rfloor$ (division entière). Vérifier `collateral_requis(100, 150) == 150`.

Question 56. Écrire `dette(P, r, n)` renvoyant $P(1+r)^n$. Montrer que la dette dépasse strictement le collatéral G pour la première fois à la période

$$n^* = \left\lfloor \frac{\ln(G/P)}{\ln(1+r)} \right\rfloor + 1 \quad (\text{si } G > P).$$

Question 57. Application : pour $P = 100$, $r = 0,10$ et $G = 150$, calculer n^* et vérifier l'encadrement `dette(100, 0.1,  $n^* - 1$ ) ≤ 150 < dette(100, 0.1,  $n^*$ )`. Justifier la **terminaison** d'une boucle **while** qui incrémenterait n tant que `dette( $P, r, n$ ) ≤  $G$` .